Learning Dynamics

Homework four

Paolo Alberto Bordis

Student ID: 000628838

Affiliation: ULB

2025/2026

Contents

Introduction **3**

1 Implementation **4**

1.1 Config 4

1.2 Player 5

1.3 Environment 6

1.4 Independent Q-learning / Sparse Q-learning algorithm 7

1.5 Training 7

1.6 Utils, Storage, and Notebook 7

2 Results **8**

2.1 Hyperparameter Settings 8

2.2 Results and Comparison 9

2.2.1 Env. 1 (2 agents) 9

2.2.2 Env. 2 (2 agents) 9

2.2.3 Env. 2 (4 agents) 9

2.2.4 Printout of the environment 13

List of Tables

2.1 Global Hyperparameters used across all experiments. 8

List of Figures

1.1 Grid representation of the Agent's Field of View. 6

2.1 Comparison of performance in Environment 1 (2 Agents). Left: Independent; Right: Sparse. . 10
2.2 Comparison of performance in Environment 2 (2 Agents). Left: Independent; Right: Sparse. . 11
2.3 Comparison of performance in Environment 2 (4 Agents). Left: Independent; Right: Sparse. . 12
2.4 This is my plot imported as a PNG. 14
2.5 This is my plot imported as a PNG. 14
2.6 This is my plot imported as a PNG. 15

Introduction

In this project, we analyze and compare the performance of agents using different reinforcement learning algorithms.

The primary focus of this study is to analyze agent behavior in scenarios where coordination within the environment is only intermittently necessary or possible. Following the implementation of the two algorithms (Independent Q-Learning and Sparse Q-Learning), we conduct a comparative analysis to determine if one algorithm strictly outperforms the other or if a trade-off exists between them.

Finally, we conclude by addressing key questions regarding the performance observed in our analysis.

1 Implementation

The implementation has been structured according to the following components:

- Config
- Player
- Environment
- Independent Q-learning / Sparse Q-learning algorithm
- Training
- Storage
- Utils
- Experiment Runner
- Notebook main

Defining these as separate classes is preferable to using a single notebook containing all the code, as it improves modularity and readability.

1.1 Config

The first class implemented is `Config.py` (or Configuration), in which I defined the grid for every environment as well as all information regarding initial positioning, goals, and collisions.

To be clear, the grid (or matrix) has been implemented as a NumPy array. The border walls are not explicitly defined in the grid itself; instead, they are handled via a check when an agent attempts to go out of bounds. This matrix is populated with numbers, where 0 defines a wall, and every other position (internal walls aside) is associated with a unique number representing the state of that position within the map.

```
# ENV 1: (21 states) - 5x5 Grid
self.grid_1 = np.array([
[1,  2,  0,  3,  4],
[5,  6,  0,  7,  8],
[9, 10, 11, 12, 13],  # 11 is the Doorway (Center)
[14,15,  0, 16, 17],
[18,19,  0, 20, 21]
])
# ENV 2: Four Rooms (36 States) - 7x7 Grid
self.grid_2 = np.array([
[0,  1,  2,  0,  3,  4,  0],
[5,  6,  7,  0,  8,  9, 10],
[11,12, 13, 14, 15, 16, 17], # Horizontal Hallway
[0,  0, 18,  0, 19,  0,  0], # Vertical connectors
[20,21, 22, 23, 24, 25, 26], # Horizontal Hallway
[27,28, 29,  0, 30, 31, 32],
[0, 33, 34,  0, 35, 36,  0]
])
```

Listing 1.1: Grid Environment Configuration

This code example shows the grid implementation. We can see that walls are identified as 0 inside the map, and for every position, we have a unique number that identifies the current state within the grid.

1.2 Player

I created the `Player` class to lighten the `Environment` class. Specifically, it defines the agent's position inside the grid (row for y and col for x). The only two critical methods are the `step` method and the `state` method.

```
slips = {
  0: [3, 2], # North (0) -> Slips West (3) or East (2)
  1: [2, 3], # South (1) -> Slips East (2) or West (3)
  2: [0, 1], # East  (2) -> Slips North (0) or South (1)
  3: [1, 0]  # West  (3) -> Slips South (1) or North (0)
}
# CLEANER APPROACH
outcomes = [action, slips[action][0], slips[action][1]]
probs = [0.8, 0.1, 0.1]
# 1 line of code. No math errors possible.
actual_move = np.random.choice(outcomes, p=probs)
```

Listing 1.2: Step action inside the Player class

Following the rules, this is how an agent actually works: usually, it attempts an action, but there is only an 80% probability of executing that specific action.

The other method is a simple setter and getter for the state, a useful trick to confirm whether an agent has truly reached the goal.

1.3 Environment

The `Environment` class follows the structure defined for the second exercise, with the same parameters defined for the output, but without truncations. I believe the environment should orchestrate the other classes (`Player` and `Config`) and provide information during training, without needing to handle excessive internal logic.

The only thing handled by the environment is the check for perception. Usually, this is not fixed to true or false for every simulation but changes dynamically over time.

I implemented this using Manhattan distance. Naturally, this depends on how you set the distance threshold. We can visualize how Manhattan distance works as follows:

```
def check_active_percept(my_pos, other_pos):
# 1. Calculate Vertical Distance (Rows)
# Example: |2 - 0| = 2 steps
row_dist = abs(my_pos[0] - other_pos[0])
# 2. Calculate Horizontal Distance (Cols)
# Example: |2 - 3| = 1 step
col_dist = abs(my_pos[1] - other_pos[1])
# 3. Manhattan Distance (Sum of absolute differences)
total_dist = row_dist + col_dist
# 4. Check if within Field of View (FOV)
return total_dist <= 2
```

Listing 1.3: How Manhattan Distance works for Active Percept

Manhattan Dist ≤ 2

·	·	2	·	·
·	2	1	2	·
2	1	**Q**	1	2
·	2	1	2	·
·	·	2	·	·

Figure 1.1: Grid representation of the Agent's Field of View.

Usually, it is better to set the distance to 2 and neither increase nor decrease it. The reason is that we do not have a giant matrix, so this Field of View (FOV) is correct; typically, we do not see better performance

by increasing the FOV because sometimes it is beneficial to see others, but other times it is better not to see anything.

1.4 Independent Q-learning / Sparse Q-learning algorithm

Here I defined two algorithms. The first one is literally the independent learner developed for the second exercise without any modifications.

The second one is slightly different because, in the Sparse algorithm, we need to define two Q-tables. The first is the classic Q-table as defined for the independent learner, while the second is a dictionary.

1.5 Training

The `Training` class is the primary class implemented to learn and evaluate the agents. It follows exactly the training main loop defined for the second exercise regarding independent learner training. However, for the sparse learner, it is slightly different because we need to obtain more data from the environment and perform other tasks. Nevertheless, the pattern remains exactly the same: we need to reset for every episode, and for every step, select an action for each agent, invoke the step inside the environment, and then update the state inside the Q-local table and Q-common table.

1.6 Utils, Storage, and Notebook

These classes are useful for handling multithreading. In particular, they have been implemented to try to reduce the time required to complete all these simulations, as we have 7 separate training runs across different environments and agents. With this implementation, I can obtain all results in about 10 to 12 minutes.

2 Results

2.1 Hyperparameter Settings

Before evaluating the models, it is essential to clarify the hyperparameters used in the experiments.

While some parameters are fixed by the assignment requirements, others were tuned to optimize training performance. The following table summarizes the specific parameterization used for each scenario. Note that these parameters are uniform across all agents within the same experiment, adhering to the project constraints.

To ensure a fair comparison, we used a uniform set of hyperparameters across all experiments (Env 1 and Env 2, for both 2 and 4 agents):

γ	α	ϵ_{max}	ϵ_{min}	**Target Decay**
0.98	Dynamic	1.0	0.15	2/3 Episodes

Table 2.1: Global Hyperparameters used across all experiments.

The decay rate, ϵ_{decay} (or λ), was calculated following the project guidelines, which state that "the decay rate should be set so epsilon decreases slowly enough towards ϵ_{min}, more or less somewhere between 2/3rds or at the end of your training episodes."

Assuming an ϵ_{min} of 0.05 and a target episode $T_{\text{target}} \approx 6666$ (two-thirds of the total 10,000 episodes), we derived the decay rate λ using the inverse exponential decay formula:

$$\lambda = \left(\frac{\epsilon_{\text{min}}}{\epsilon_{\text{max}}}\right)^{\frac{1}{T_{\text{target}}}} \tag{2.1}$$

Substituting the values:

$$\lambda = \left(\frac{0.05}{1.0}\right)^{\frac{1}{6666}} \approx 0.99955 \tag{2.2}$$

This value ensures that the agents maintain significant exploration during the critical early learning phase before converging to a greedy policy.

2.2 Results and Comparison

2.2.1 Env. 1 (2 agents)

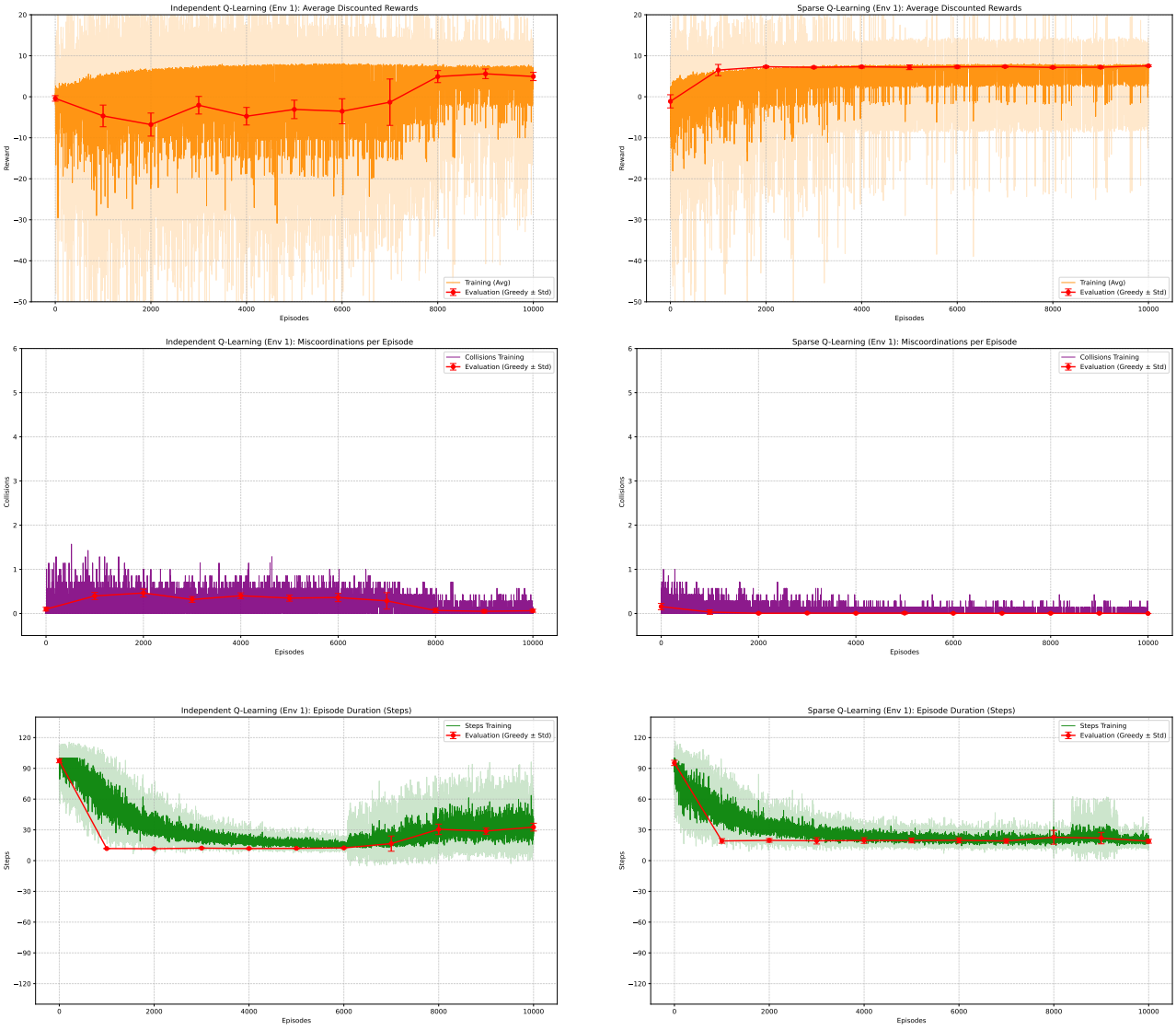
The first result is quite clear: the Independent learner performs worse than the Sparse learner. During evaluation, the Sparse learner is able to reach the reward levels seen in training and exhibits fewer miscoordinations than the Independent one. As mentioned previously, this trend is also observed in the step count, showing good performance after at least 6000 episodes, whereas the Independent agent performs poorly until the end of training. We can also observe how the Independent agents, being unable to coordinate during evaluation, accrue higher penalties compared to the Sparse interaction learners.

2.2.2 Env. 2 (2 agents)

In the second scenario, the Sparse learner outperformed the Independent learner. This is likely because, even with a larger map than the first environment, the Sparse mechanism was able to successfully coordinate the two agents. Initially, during training, we observe a high standard deviation for both algorithms; however, across the episodes, we see very good performance in terms of steps and other metrics. As noted in the previous environment, the Independent learner incurred more penalties than the Sparse interaction learners.

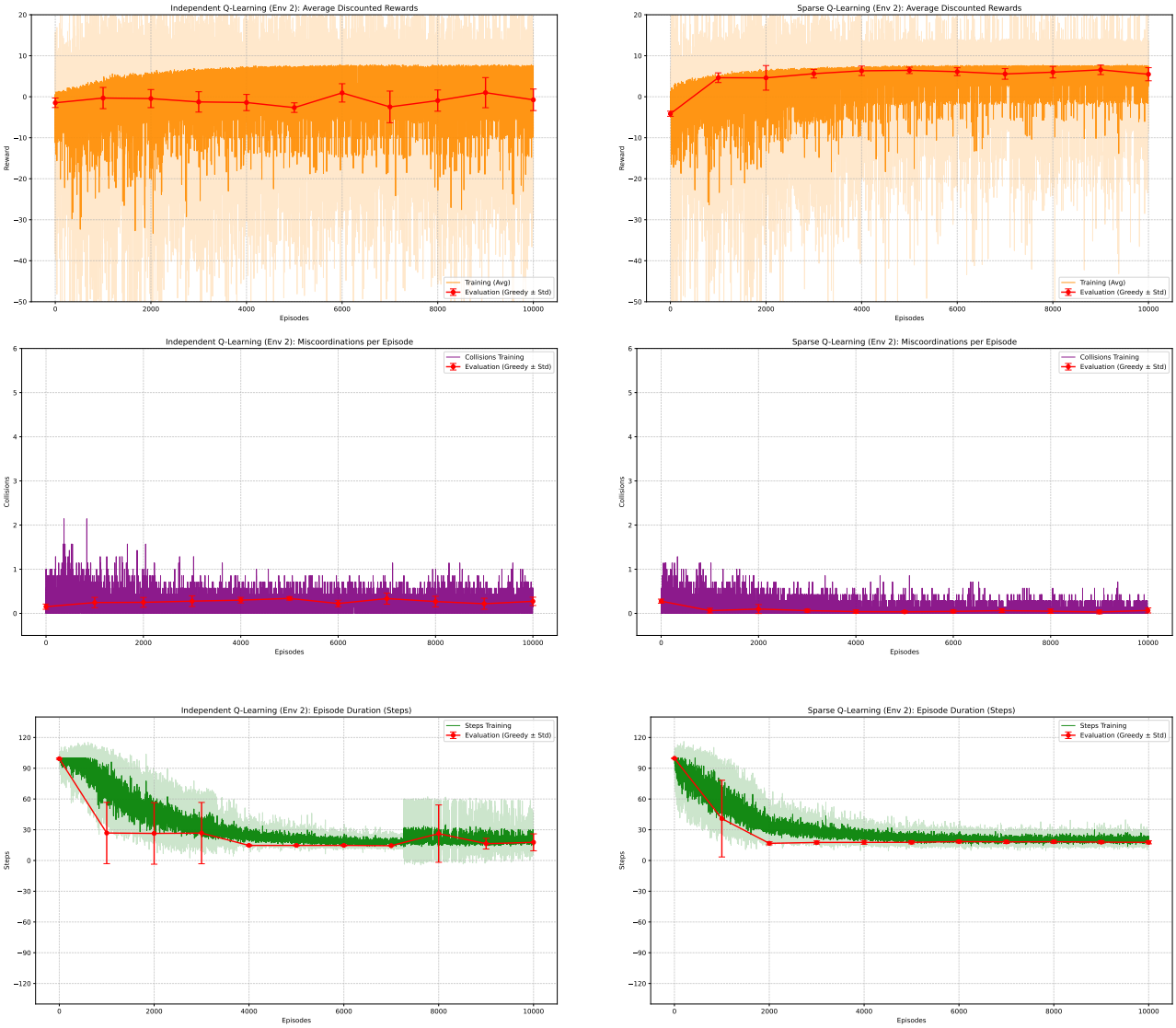
2.2.3 Env. 2 (4 agents)

This experiment differs from the previous two because there are more agents within the map, which impacts the performance stability of both algorithms. This experiment exhibits high randomness. In this specific training run, a solution was reached, allowing for comparison; however, obtaining a robust assessment of whether one algorithm is strictly better than the other usually requires averaging over multiple runs. According to the plots below, we can state that the Sparse learner demonstrates better performance than the Independent learner. The performance is good because, in this run, the agents did not develop "paranoia" regarding the doorways. This phenomenon typically occurs when agents become afraid of entering doorways due to collision penalties during training; they attempt to minimize miscoordination by avoiding the door, but consequently fail to reach the goal. Regarding the Independent learner, it appears less hesitant to explore new states despite receiving penalties. In fact, from approximately episode 5000 to 7000, the standard deviation is very low, but it increases from episode 7000 until the end of training. Conversely, the Sparse interaction learner aims to minimize miscoordination while actively trying to reach the goal, especially in the final episodes. This explains why, at the tail end of the training, the step count decreases significantly.



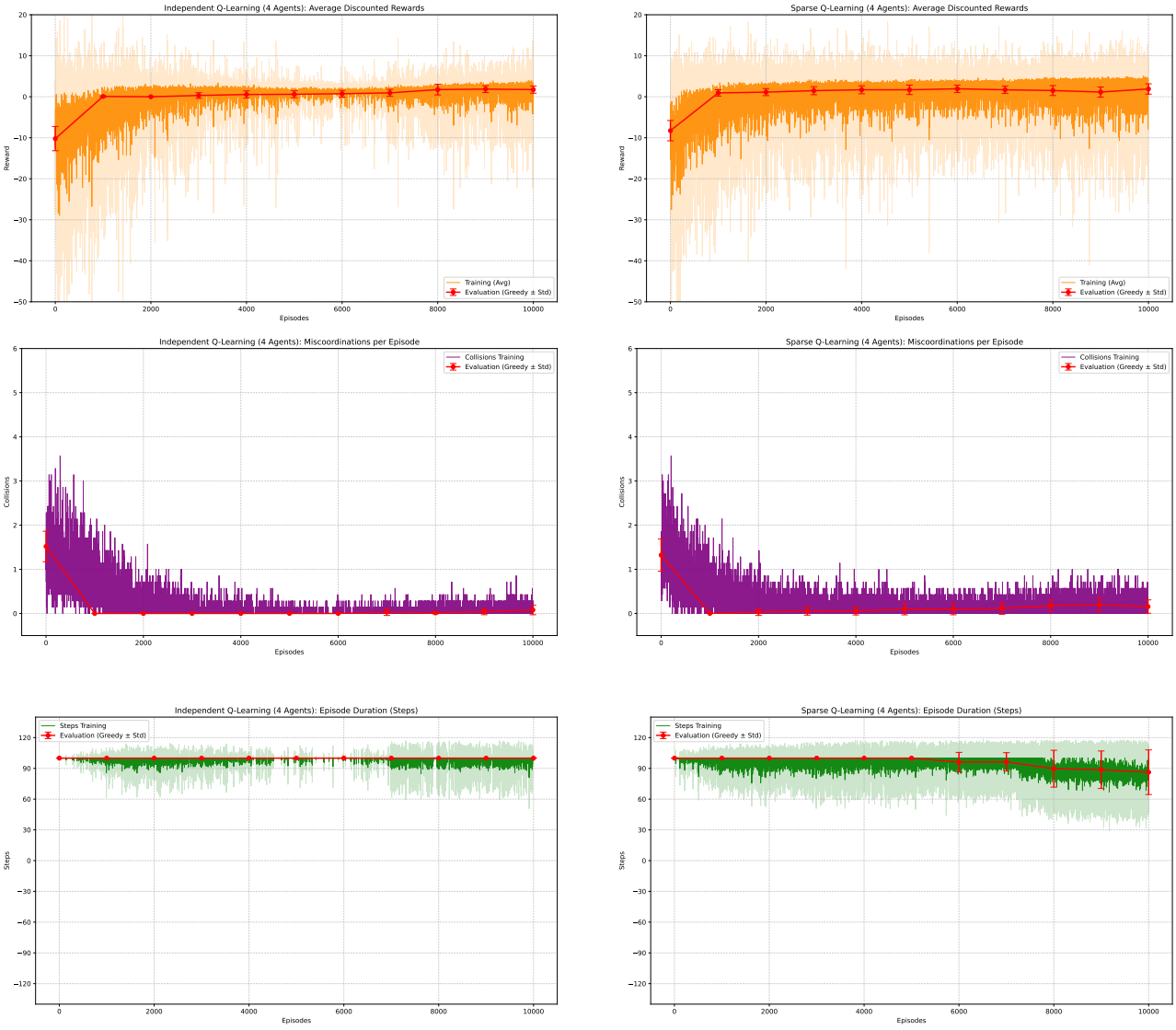
Independent Q-Learning Sparse Q-Learning

Figure 2.1: Comparison of performance in Environment 1 (2 Agents). Left: Independent; Right: Sparse.



Independent Q-Learning Sparse Q-Learning

Figure 2.2: Comparison of performance in Environment 2 (2 Agents). Left: Independent; Right: Sparse.



Independent Q-Learning Sparse Q-Learning

Figure 2.3: Comparison of performance in Environment 2 (4 Agents). Left: Independent; Right: Sparse.

2.2.4 Printout of the environment

We can confirm the emergence of asymmetry by analyzing the step-by-step plot of the environment. Typically, we observe that one agent positions itself within or near the doorway and waits to observe the other agent's movement. Meanwhile, the second agent, recognizing that the doorway is occupied, chooses not to pass immediately but instead waits for the first agent's next action, thereby facilitating coordination.



Figure 2.4: This is my plot imported as a PNG.

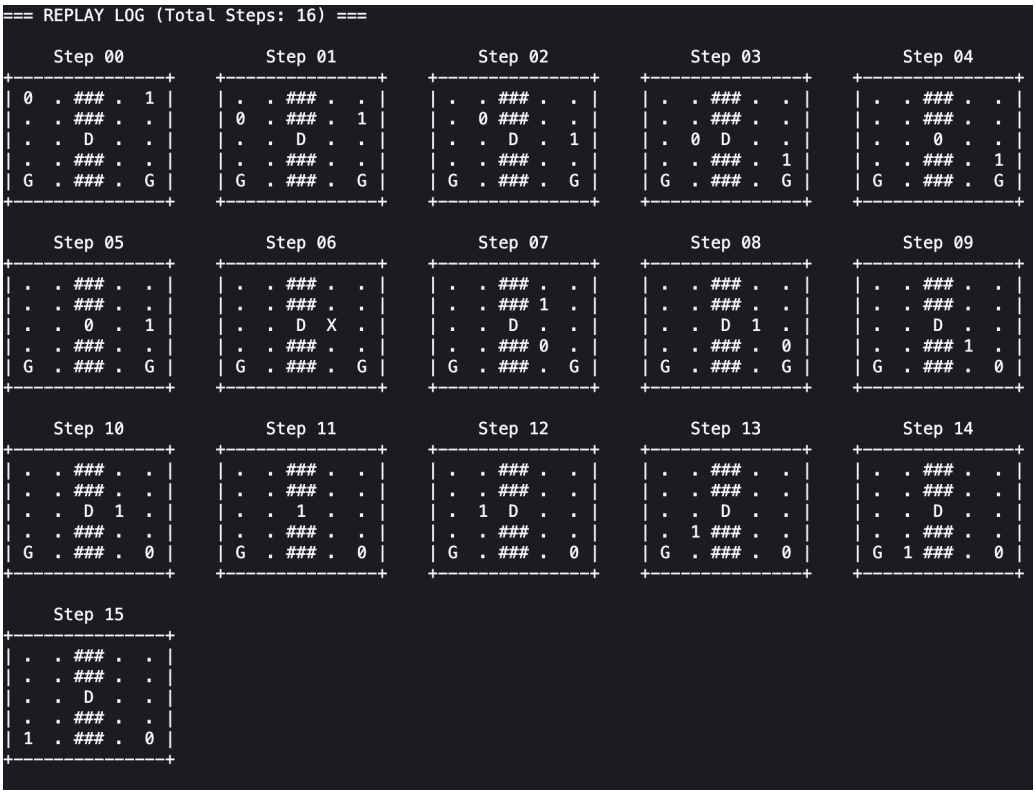


Figure 2.5: This is my plot imported as a PNG.



Figure 2.6: This is my plot imported as a PNG.